

AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design

Midterm Defense

Yujie Feng

Supervisor: Jin Hou

- ① Project Overview
- ② Progress Summary
- ③ Module A: Bird Nest Detection (Completed)
- ④ Module B: Oil Leak Detection (Trained & Deployed)
- ⑤ Module C: Infrared High-Temperature Detection (Integrated)
- ⑥ SaaS Flask Microservice Platform
- ⑦ Problems & Challenges
- ⑧ Next Phase Work Plan

Project Overview: A Scalable SaaS Inspection Platform

AI Platform Core

- SaaS backend
 - unified auth / RBAC
 - multi-tenant isolation
 - task queue / data lake
- AI model registry
 - Module A: Bird Nest Detection (Done)
 - Module B: Oil Leak Detection (In progress)
 - Module C: Infrared High-Temp. (Integrated)

Multi-modal Inputs

- UAV RGB imagery
- Infrared imagery
- Sensor telemetry streams
- Historical inspection records

Enterprise Outputs

- Alert center & dashboard
- Maintenance work orders
- Visual analytics portal
- API & mobile integration

Midterm Progress Summary

Strategy: “Vertical Slice First, then Horizontal Scale”

Complete one module end-to-end before extending to multi-module registry and unified scheduling.

Module A (Completed)

- Mask R-CNN instance segmentation
- End-to-end PoC: data $\rightarrow$ training $\rightarrow$ deployment
- bbox $AP_{50} = 1.000$, segm $AP_{50} = 1.000$
- Flask microservice deployed

Module B (In Progress)

- YOLOv8s object detection
- Trained on CSDN dataset (123 epochs)
- Best mAP50 ≈ 0.39
- Integrated with Flask API

Module C (Integrated)

- OCR + pixel-mapping hybrid
- No dedicated DL model training
- User-defined alarm temperature
- Hotspot visualization deployed

Dataset & Augmentation

- 54 COCO-format images (43 train, 11 val)
- Bounding boxes + instance segmentation polygons
- Deterministic augmentation pipeline:
 - Custom Random IoU Crop (scale 0.3–1.0)
 - Photometric jitter (brightness, contrast)
 - ResizeMax + PadSquare (512×512)
- *Why deterministic? Generative augmentation introduced physically implausible artifacts.*

Model & Training Config

- Model: Mask R-CNN (ResNet50 + FPN v2)
- Input: 512×512 , batch size 4, 40 epochs
- Optimizer: AdamW ($\text{lr}=5 \times 10^{-4}$) + OneCycleLR
- Mixed precision (AMP) enabled
- ~ 27 s per epoch; GPU utilization 70–85%

Module A: Bird Nest Detection — Validation Metrics

COCO Validation Metrics

Metric	bbox	segm
AP@[0.50:0.95]	0.805	0.795
AP50	1.000	1.000
AP75	1.000	0.919
AR@[0.50:0.95]	0.837	0.828

Score threshold 0.4–0.5 gives optimal F1 trade-off in deployment.

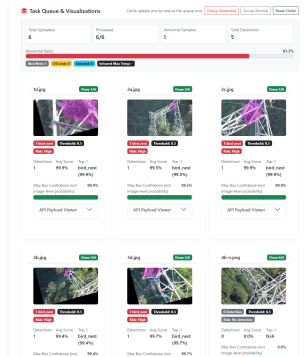


Figure: Bird nest detection: bounding box and instance mask overlay.

Module A: Bird Nest Detection — Training Dynamics

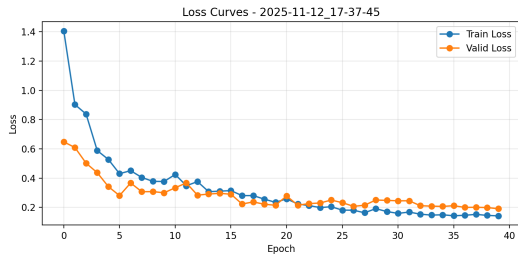


Figure: Train/Validation Loss Curves

Key Observations

- Train loss: $1.41 \rightarrow 0.14$; Val loss: $0.65 \rightarrow 0.19$
- Loss plateaus around epoch 20 — no overfitting
- Right-skewed IoU histogram: strong localization
- PR curve stays near precision 1.0 at IoU=0.50
- IoU=0.75 settles ≈ 0.93 ; IoU=0.90 dips to ≈ 0.70

Module A: Bird Nest Detection — PR & Threshold Analysis

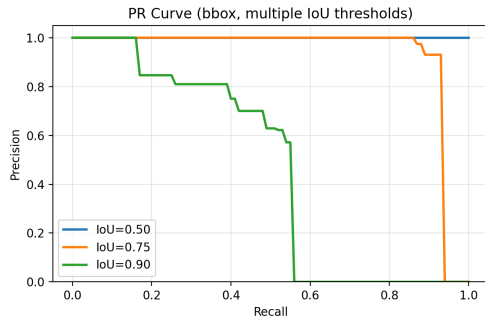


Figure: Bbox PR Curves (IoU=0.50/0.75/0.90)

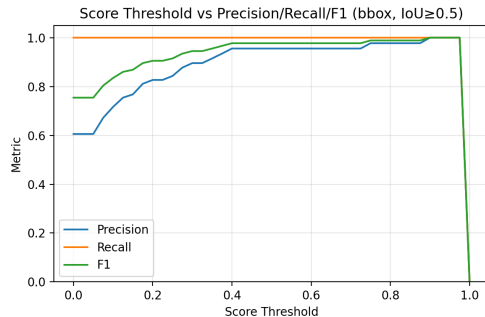


Figure: Score Threshold vs. P/R/F1 (IoU $\geq$ 0.5)

With $AP_{50}=1.0$, threshold 0.4–0.5 optimizes F1; higher thresholds (0.6–0.8) trade recall for precision in deployment.

Data Sources

- Original VOC-format dataset (~103 images, XML annotations)
- YOLO-format CSDN dataset
- `prepare_dataset.py`: VOC-only / CSDN-only / merged (~333 images, 8:2 split)
- Single class: `oil_leak`
- Results reported from **train10-csdn** (CSDN dataset only)

Training Configuration (YOLOv8s)

- Model: `yolo8s.pt`, single-class detection
- 150 epochs, input 640, batch 8
- AdamW optimizer, cosine LR schedule
- Mosaic 0.3, mixup 0.05, RAM cache
- Patience 30, AMP enabled, seed 42

Module B: Oil Leak Detection — Validation Metrics

Best Validation Results (train10-csdn, epoch 93–94)

Metric	Value
Precision(B)	≈ 0.83
Recall(B)	≈ 0.34
mAP50(B)	$\approx \mathbf{0.39}$
mAP50-95(B)	$\approx \mathbf{0.26}$
Total epochs run	123

High precision / low recall suggests conservative detections; full merged-dataset training planned next.

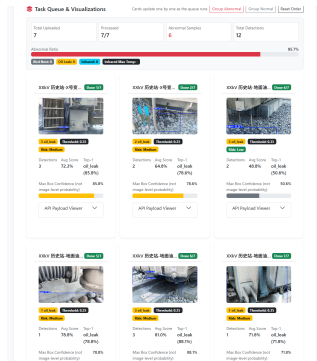


Figure: Oil leak detection: identified leakage region on equipment.

Module B: Oil Leak Detection — Training Results

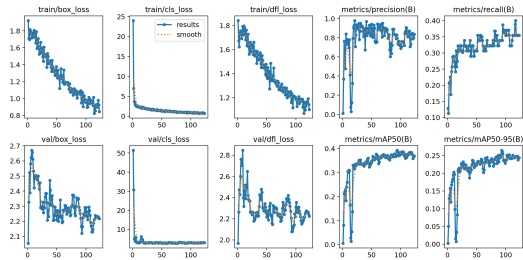


Figure: Training Results (loss / mAP curves)

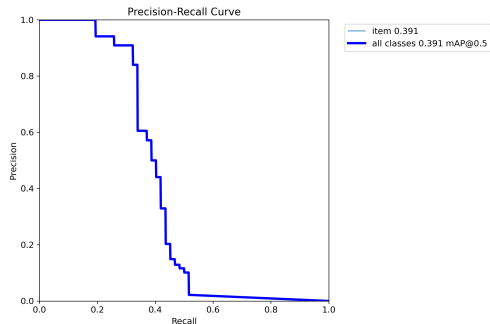


Figure: Validation PR Curve (IoU 0.5)

Module B: Oil Leak Detection — F1 & Confusion Matrix

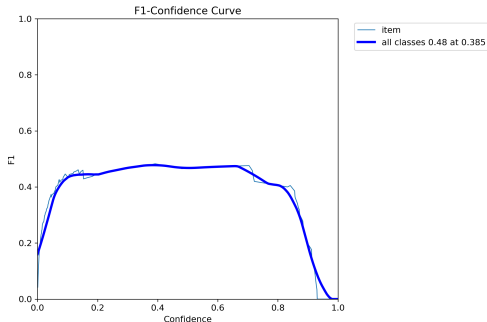


Figure: Validation F1 Curve

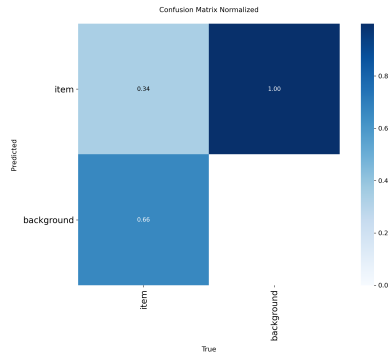


Figure: Normalized Confusion Matrix

Box-level detection sufficient for “presence and approximate location”; pixel-level segmentation can be added if area statistics are required.

Module C: Infrared High-Temperature Detection

Approach: OCR + Pixel-Mapping Hybrid

- No dedicated DL model training required
- EasyOCR parses high/low temperature scale from legend
- Pixel intensity $\rightarrow$ temperature mapping with heuristic OCR correction
- User-defined alarm temperature threshold (default 50°C)
- Outputs: hotspot boxes, per-hotspot temp., global max temp., annotated visualization

Suppression Techniques

- **Border watermark mask:** detects and masks bright white logo/watermark near image border
- **OCR text-region mask:** excludes legend and scale digits to reduce false positives
- Filters regions above alarm threshold only

Module C: Infrared Detection — Result

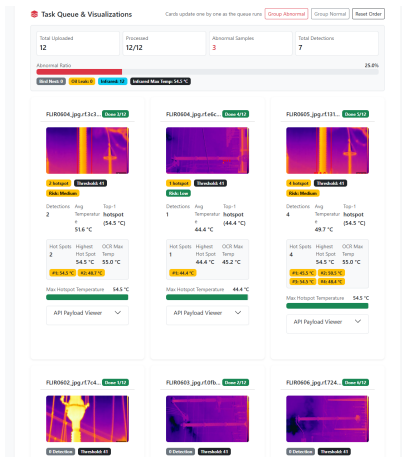


Figure: Hotspot bounding above user-defined threshold.

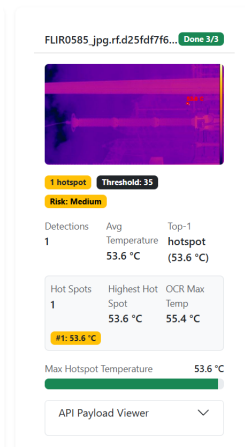


Figure: Infrared detection result card.

Flask REST API Features

- Batch image upload, on-demand inference
- Model selection: Bird Nest / Oil Leak / Infrared
- JSON result (boxes, confidence/temperature, class) + visualization
- Single & ZIP download for offline archiving
- Statistical dashboards for inference history
- Large-image handling: resize / tiling to avoid OOM

Architecture & SaaS Role

- All three modules share one upload/inference/download API
- Configurable model registry: weight path managed per model type
- “Model-as-a-Service” foundation for multi-tenant RBAC
- Fallback to local checkpoint if registry entry missing
- Structured exception handling at API layer

SaaS Platform — User Interface

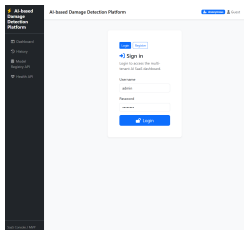


Figure: Login Interface

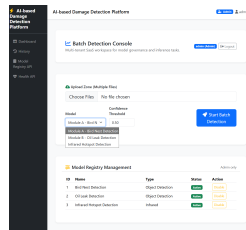


Figure: Main Dashboard & Task Flow

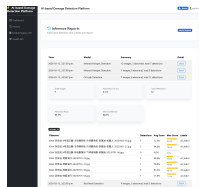


Figure: History & Statistical Analytics for Inference Records and Work Orders

Problems & Challenges

1. Generalization

- Bird nest $AP_{50}=1.0$ on same-distribution validation — may drop in real conditions (haze, backlight, new tower types, seasonal variation)
- Cross-scene data, negative and hard-sample design needed as platform-level requirements

2. Technical Choices

- **SaaS backend**: feature-complete prototype (RBAC, task queue) vs. production-grade auth (OAuth2, audit logs) — current preference: demonstrable multi-model prototype first
- **Oil leak**: YOLOv8 box-level sufficient for location; pixel-level segmentation (U-Net) needed only if area statistics required
- **Infrared**: standalone module sufficient now; RGB+heatmap fusion possible for multi-modal analysis

3. Data & Deployment Constraints

- Dataset size modest (54 bird nest; ~333 merged oil leak); annotation quality critical

Next Phase Work Plan

Track 1 — SaaS Platform (Weeks 5–10)

- Multi-tenant backend with RBAC and task orchestration
- AI model registry API: register, query, enable/disable, weight-path management
- Productize PoC training utilities: configurable augmentation, logging, checkpoint management

Track 2 — New Modules (Weeks 11–18)

- Complete Module B (oil leak) on merged dataset; update registry and metrics
- Module C (infrared): robustness and multi-device adaptation
- Reuse `prepare_dataset`, training scripts, deployment pipeline for new modules

Weeks 19–20: Final Integration & Defence

- End-to-end tests for multi-module and multi-user scenarios
- Consolidate experiment data and screenshots; complete final report and defence materials
- Deliverable: demonstrable three-module (Bird Nest / Oil Leak / Infrared) SaaS prototype

- ① **Generalization strategy:** Should the project invest in cross-scene data collection or focus on augmentation and hard-sample mining to improve robustness of all three modules?
- ② **Oil leak improvement:** Current mAP50 ≈ 0.39 on CSDN dataset only. Should training be re-run on the merged VOC+CSDN dataset, or should the model architecture be upgraded (e.g. YOLOv8m / segmentation head)?
- ③ **SaaS depth vs. breadth:** Should the final phase prioritize a fully functional RBAC and task-queue backend, or focus on expanding the AI modules and demo quality for the defence?